Calepin

What is Calepin?

Typst

Typst is a modern typesetting system that compiles plain text markup into rich documents. Think of it as a simple Markdown-like syntax, with the scriptability of LaTeX, and the ability to export to PDF, SVG, and HTML files. Typst is ultra fast, easy to learn, and expressive, which makes it a comfortable tool for anything from short letters to full scientific manuscripts.

Computational notebooks

A computational notebook mixes prose with executable code. Figures, tables, and numbers are computed when the document is rendered, rather than pasted-in by hand. This allows analysts to create reproducible reports in the tradition of literate programming.

Some notebook tools can act as frontends for Typst, but they often superimpose their own syntax, language, and structure. For example, Posit's Quarto supports an extended idiom of Markdown, which can be translated to Typst and then rendered as PDF.

Calepin

Calepin is a Typst-native computational notebook. Rather than hide Typst behind Markdown, it embeds executable code chunks directly inside `.typ` documents. *Calepin* scans a `.typ` file for inline code or exectuable chunks, evaluates them, and lets Typst render the final document with computed results in place.

Calepin is language-agnostic. It can execute code in Python, R, and any language with a Jupyter kernel. It can also render diagrams using engines like Mermaid, Graphviz, TikZ, and D2.

Calepin comes with extensions for popular editors like VS-Code, Cursor, and Positron (via the Microsoft and VSX marketplaces).

- VS Code: VincentArel-Bundock.calepin
- Open VSX: VincentArel-Bundock/calepin

Render

Use `calepin compile` when you want to execute code chunks and render the notebook. The command line shape is intentionally the same as `typst compile`: same output-first/format-driven arguments, with `--` pass-through for Typst flags, plus Calepin preprocessing.

```
calepin compile paper.typ --format pdf
calepin compile paper.typ --format html
calepin compile paper.typ {p}.svg --format svg

# explicit output path
calepin compile paper.typ path/to/paper.pdf --format pdf
```

Arguments after `--` are forwarded to Typst, so project-specific Typst flags can stay in the same command.

```
# open pdf in system viewer
calepin compile paper.typ -- --open
```

```
# set path to font directory
calepin compile paper.typ -- --font-path fonts
```

Watch

Use `calepin watch` while editing. It watches your source for changes, re-runs preprocessing, and delegates recompilation and previewing to `typst watch`. The command form is the same as `typst watch`: same positional arguments and pass-through flags, with Calepin running its preprocessing step first.

```
calepin watch paper.typ
calepin watch paper.typ --format html
calepin watch paper.typ {p}.svg --format svg

Stop a running watch from the same project.
calepin stop paper.typ
```

The default output format is PDF. Choose another format with `--format`, or let the output extension select the format.

Arguments after `--` are passed through to `typst watch`. Typst's `--open` flag opens the rendered output in the operating system's default viewer, and Typst's `--port` flag chooses the HTML preview port.

```
calepin watch example.typ -- --open
calepin watch paper.typ paper.html --format html -- --port 3001 --open
```

PDF viewer auto-refresh

Some PDF viewers do not automatically refresh a document when it is regenerated on disk. For example, macOS Preview may keep showing an older PDF until the window is focused, the file is reopened, or the application is restarted.

For smoother live preview, use a PDF viewer that reloads the file when it changes. On macOS, Skim is a good option. Other platforms have similar auto-reloading viewers, which are useful when working with tools that repeatedly rebuild PDFs.

Example

Every document uses the runtime file *Calepin* writes to `.calepin/calepin.typ`. `calepin compile` and `calepin watch` run code chunks first, so the runtime and computed outputs are ready when Typst builds the document.

Preamble

Start by importing the runtime and setting document-wide defaults with `calepin.setup()`.

```
#import ".calepin/calepin.typ"

#calepin.setup(
  echo: true,
  eval: true,
)
```

Define a short alias for Python inline computation.

```
#let py = calepin.inline.with("python")
```

Chunks

A block chunk runs a piece of code and inserts its result. Start with a plain fenced block:

```
```python
x = 41
print(x + 1)
```

Variables are persistent across chunks:

```python
print(x + 2)
```
]
```

When you need extra control, use `#calepin.chunk` with options such as labels, captions, hiding source code, or changing how results are shown. If the body is a fenced block with a language, `#calepin.chunk` infers the engine from the fence:

```
#calepin.chunk(label: "answer")[
```python
x = 41
print(x + 1)
```
```

Inline

An inline expression drops a computed value into the surrounding prose. It uses the same raw body contract and never takes a label.

```
The inline answer is #py[`print(40 + 2)`].
```

All together now

Here is one full document example.

```
#import ".calepin/calepin.typ"

#calepin.setup(
  echo: true,
  results: "verbatim",
)

#let py = calepin.inline.with("python")

```python
x = 41
print(x + 1)
```

Variables are persistent across chunks:

```python
print(x + 2)
```

The inline answer is #py[`print(40 + 2)`].
```